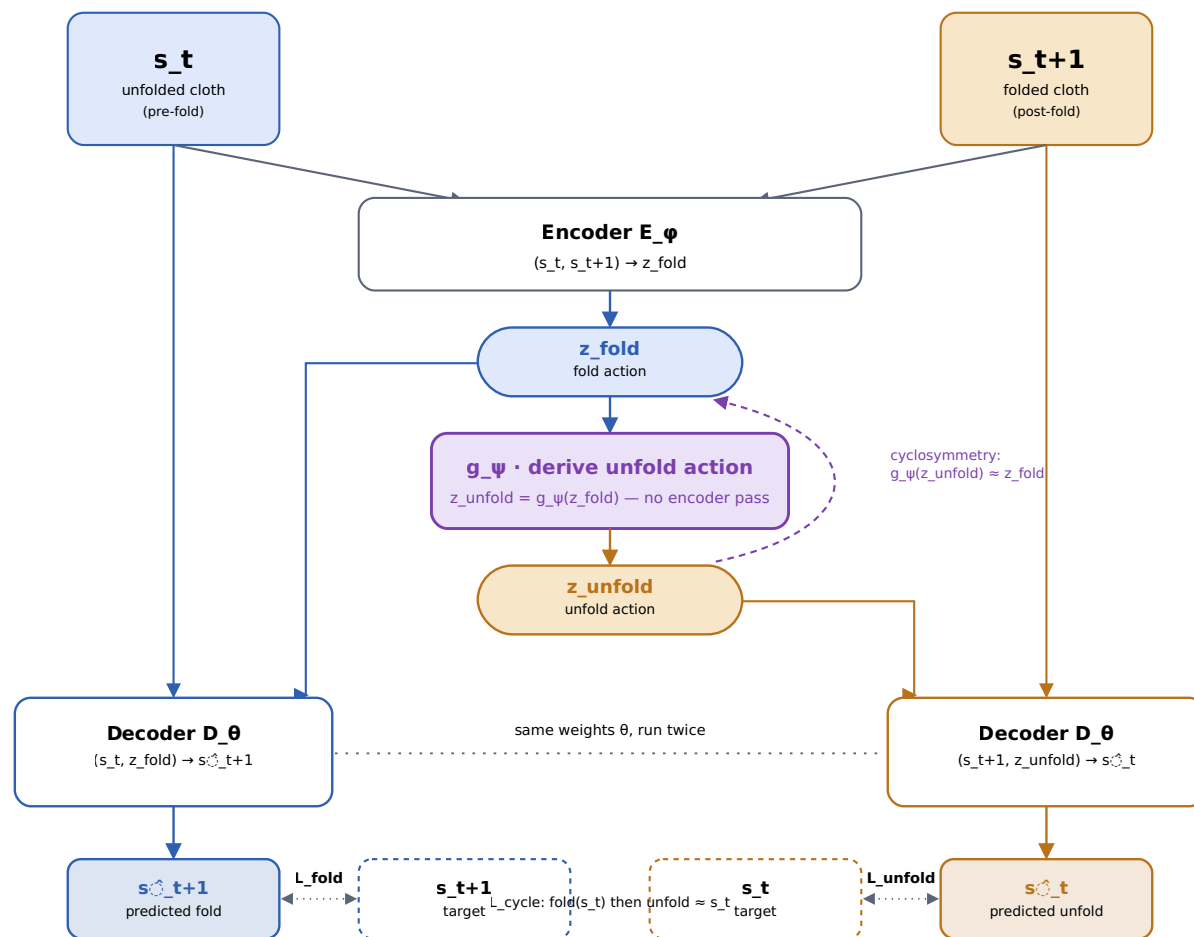


Bidirectional Latent Action Model

The input is a pair of cloth states connected by a **folding action** and its inverse, an **unfolding action**. A single encoder infers the fold action `z_fold`; a small operator derives the unfold action `z_unfold` from it, a relationship enforced by **cyclosymmetry**. A shared decoder run in both directions is tied together by **cycle-consistency**: folding then unfolding a cloth should return it to where it started. The two learned actions are finally distilled into a single **learned policy** that outputs real manipulator actions.



s_t (unfolded) and s_{t+1} (folded) feed one encoder, which outputs the fold action z_{fold} . g_ψ derives the unfold action z_{unfold} from it directly — no second encoder pass over the reversed pair. **Cyclosymmetry** holds g_ψ to an involution: running it on z_{unfold} should land back on z_{fold} . The same decoder D_θ runs twice with shared weights to produce both reconstructions, and **cycle-consistency** (L_{cycle}) requires that composing the two — folding, then unfolding — returns to the original state s_t .

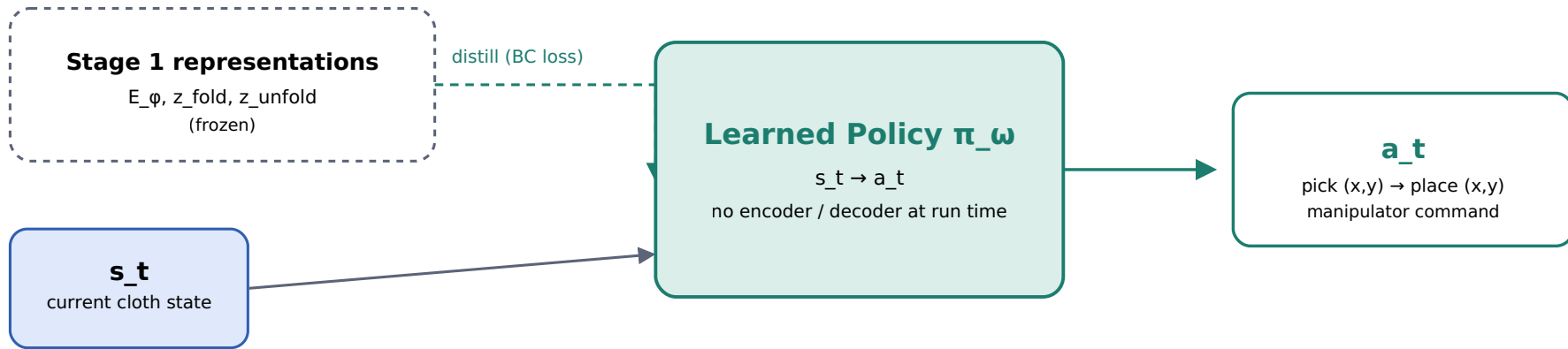
● L_{fold} $\|\hat{s}_{t+1} - s_{t+1}\|^2$

● L_{unfold} $\|\hat{s}_t - s_t\|^2$

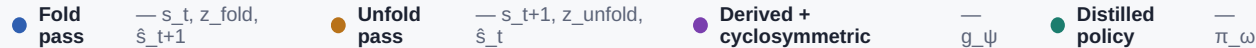
● $L_{cyclosym}$ $\|g_\psi(z_{unfold}) - z_{fold}\|^2$

● L_{cycle} $\|D_\theta(D_\theta(s_t, z_{fold}), z_{unfold}) - s_t\|^2$

STAGE 2 · POLICY DISTILLATION (DISTILLED FROM Z_FOLD, Z_UNFOLD VIA BEHAVIOR CLONING)



Once trained, **z_{fold}** and **z_{unfold}** serve as supervision for a lightweight policy **π_ω** : given the live cloth state **s_t** , it outputs a real manipulator action **a_t** directly. The encoder and decoder from Stage 1 are only needed to *train* **π_ω** — the deployed robot runs the policy alone.



Why this design

- **Half the encoder cost.** One E_ϕ pass produces z_{fold} ; there is no second $(s_{t+1}, s_t) \rightarrow z_{\text{unfold}}$ encoder pass to train or run.
- **Cyclosymmetry constrains the derivation.** g_ψ isn't free to map z_{fold} anywhere — running it again on z_{unfold} must return z_{fold} , so fold and unfold sit as mirror points of one action space rather than two unrelated ones.
- **Cycle-consistency closes the loop on states.** Independently of what the latent actions look like, composing the two decodes — fold, then unfold — has to land back on s_t . This catches drift that a purely latent constraint could miss.
- **The policy is what actually deploys.** Stage 1 only shapes a clean, cyclosymmetric action space; Stage 2 distills it into π_ω , a policy mapping cloth state directly to a manipulator command with no encoder or decoder on the critical path.